

Instituto Politécnico Nacional

*Unidad Profesional Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas*

Ingeniería Telemática
Bases de Datos Distribuidas
Prof. Carlos de la Sosa Cruz
Grupo 3TM2

Tarea Diseño
Fragmentación

Integrantes:

Nieves Hernandez Efrain
Rodríguez Escalera Ricardo
Sánchez Lemus Ulises Mariano

Production.Product tiene una relación jerárquica muy clara con las tablas de ventas, lo que facilita enormemente la aplicación del algoritmo de fragmentación derivada (semi-join) sin enredarse en dependencias circulares.

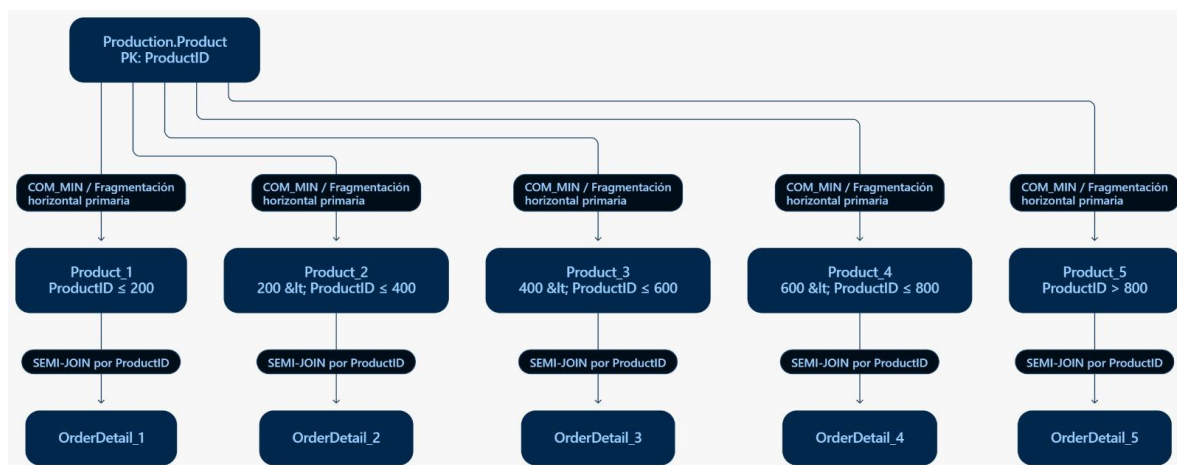
1. Modelo de Datos y Grafo Relacional

Para descentralizar el catálogo y optimizar el inventario local, tomaremos la tabla de productos como la entidad central y propagaremos esa fragmentación hacia los detalles de las órdenes de venta, ya que estos dependen directamente del producto consultado.

- **Tabla Propietaria (A):** Production.Product
- **Tabla Miembro (C):** Sales.SalesOrderDetail (Depende de Product a través de ProductID)

El segmento del grafo relacional queda definido de la siguiente manera:

Production.Product (Propietaria) → **Sales.SalesOrderDetail** (Miembro)



2. Fragmentación Horizontal Primaria (COM_MIN)

El algoritmo COM_MIN exige que el conjunto de predicados simples sea **completo** (toda fila debe pertenecer a algún fragmento) y **mínimo** (ningún predicado debe ser redundante).

Asumiendo que tras correr el script del profesor descubrimos que los ProductID van del 1 al 1000, dividimos equitativamente la carga en 5 nodos regionales mediante predicados de rango mutuamente excluyentes.

Conjunto de Predicados (PR):

- $p_1: ProductID \leq 200$ (Nodo 1)
- $p_2: ProductID \leq 400$ (Nodo 2)
- $p_3: ProductID \leq 600$ (Nodo 3)
- $p_4: ProductID \leq 800$ (Nodo 4)
- $p_5: ProductID > 800$ (Nodo 5)

Predicados primos PR'

$$P(\text{Product}, PR') = \{F1, F2, F3, F4, F5\}$$

Donde

$$F_1 = \sigma_{p1}(\text{Product})$$

$$F_2 = \sigma_{p2}(\text{Product})$$

$$F_3 = \sigma_{p3}(\text{Product})$$

$$F_4 = \sigma_{p4}(\text{Product})$$

$$F_5 = \sigma_{p5}(\text{Product})$$

Algoritmo P de Fragmentación Horizontal

El algoritmo *P* aplica cada predicado de PR' mediante selección sobre la relación propietaria *Production.Product*:

$$P(\text{Product}, PR') = \{ \sigma_{p1}(\text{Product}), \sigma_{p2}(\text{Product}), \sigma_{p3}(\text{Product}), \sigma_{p4}(\text{Product}), \sigma_{p5}(\text{Product}) \}$$

Obteniéndose los cinco fragmentos horizontales:

$$\text{Product}_1, \text{Product}_2, \text{Product}_3, \text{Product}_4, \text{Product}_5$$

Dado que los rangos abarcan la totalidad del espectro de IDs sin superponerse, el conjunto cumple con COM_MIN. Los fragmentos primarios generados son:

- $\text{Product}_1 = \sigma_{\text{ProductID} \leq 200}(\text{Product})$
- $\text{Product}_2 = \sigma_{\text{ProductID} \leq 400}(\text{Product})$
- $\text{Product}_3 = \sigma_{\text{ProductID} \leq 600}(\text{Product})$
- $\text{Product}_4 = \sigma_{\text{ProductID} \leq 800}(\text{Product})$
- $\text{Product}_5 = \sigma_{\text{ProductID} > 800}(\text{Product})$

Predicados Minitérminos Válidos (M):

- m_1 (Fragmento 1): $p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge \neg p_4 \wedge \neg p_5$

(Representa a los productos cuyo ID es menor o igual a 200, negando pertenecer a los demás rangos).

- m_2 (Fragmento 2): $\neg p_1 \wedge p_2 \wedge \neg p_3 \wedge \neg p_4 \wedge \neg p_5$

(Representa a los productos entre 201 y 400).

- m_3 (Fragmento 3): $\neg p_1 \wedge \neg p_2 \wedge p_3 \wedge \neg p_4 \wedge \neg p_5$

(Representa a los productos entre 401 y 600).

- m_4 (Fragmento 4): $\neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge p_4 \wedge \neg p_5$

(Representa a los productos entre 601 y 800).

- m_5 (Fragmento 5): $\neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge \neg p_4 \wedge p_5$

(Representa a los productos mayores a 800).

3. Fragmentación Derivada (SEMI-JOIN)

Para que cada nodo regional consulte sus órdenes de forma autónoma sin saturar el servidor central, aplicamos una fragmentación derivada a la tabla miembro (Sales.SalesOrderDetail). Cada fragmento derivado ($OrderDetail_i$) contendrá únicamente los detalles de órdenes que correspondan a los productos de su respectivo nodo ($Product_i$).

La operación en álgebra relacional es:

$$OrderDetail_i = SalesOrderDetail \bowtie_{ProductID} Product_i$$

4. Validación del Diseño (Teoría de M. Tamer Özsu)

Para que el diseño sea teóricamente correcto, verificamos el cumplimiento de las 3 reglas fundamentales de fragmentación:

- **Complejitud (Completeness):** Se cumple. Todo ProductID existente en la base de datos caerá inevitablemente en uno de los 5 rangos lógicos definidos (desde ≤ 200 hasta > 800). Ningún producto queda huérfano. Del mismo modo, todo registro en SalesOrderDetail tiene un ProductID válido, por lo que obligatoriamente se asignará a un fragmento derivado.
- **Reconstrucción (Reconstruction):** Se cumple. La tabla original Production.Product puede ser reconstruida en su totalidad aplicando el operador de Unión ($Product_1 \cup Product_2 \cup \dots \cup Product_5$). La tabla SalesOrderDetail se reconstruye de la misma manera uniendo sus 5 fragmentos derivados.
- **Disyunción (Disjointness):** Se cumple. Los predicados utilizan operadores estrictos ($<$ y $\leq$) que crean fronteras matemáticas cerradas. Es imposible que un mismo producto exista simultáneamente en el Nodo 1 y en el Nodo 2.

Este diseño permite que el 100% de las operaciones de inventario, actualización de stock y consulta de ventas por producto se resuelvan localmente en cada nodo regional. Al fragmentar el catálogo

por rangos y arrastrar el histórico de ventas (SalesOrderDetail) mediante semi-joins, logramos aislar el tráfico de red y eliminar por completo la dependencia de un servidor central monolítico.